

SAVITRIBAI PHULE PUNE UNIVERSITY

A MINI PROJECT REPORT ON

Design and Development of a Browser History Forensic Analysis Tool

SUBMITTED TOWARDS THE
PARTIAL FULFILLMENT OF THE REQUIREMENTS OF
LABORATORY PRACTICE IV (CYBER SECURITY AND DIGITAL FORENSICS)

BACHELOR OF ENGINEERING (Computer Engineering)

BY

Vishwajeet Tulse	Exam No: _____
Student Name 2	Exam No: _____
Student Name 3	Exam No: _____
Student Name 4	Exam No: _____

Under The Guidance of

Prof. Guide Name



Sinhgad Institutes
DEPARTMENT OF COMPUTER ENGINEERING
NBN SINHGAD SCHOOL OF ENGINEERING
10/1, AMBEGAON (BK), PUNE-411041

2026-27

Sinhgad Institutes

NBN SINHGAD SCHOOL OF ENGINEERING
DEPARTMENT OF COMPUTER ENGINEERING

CERTIFICATE

This is to certify that the Mini Project Entitled

Browser History Forensic Analysis Tool

Submitted by

Vishwajeet Tulse	Exam No: _____
Student Name 2	Exam No: _____
Student Name 3	Exam No: _____
Student Name 4	Exam No: _____

is a bonafide work carried out by Students under the supervision of Prof. Guide Name and it is submitted towards the partial fulfillment of the requirement of Bachelor of Engineering (Computer Engineering) for the course Laboratory Practice IV.

Prof. Guide Name
Internal Guide
Dept. of Computer Engg.

Dr. S. P. Bendale
H.O.D
Dept. of Computer Engg.

Dr. S. P. Patil
Principal
NBN Sinhgad School of Engineering

Signature of Internal Examiner

Signature of External Examiner

PROJECT APPROVAL SHEET

Design and Development of a Browser History Forensic Analysis Tool

Is successfully completed by

Vishwajeet Tulse	Exam No: _____
Student Name 2	Exam No: _____
Student Name 3	Exam No: _____
Student Name 4	Exam No: _____

at

DEPARTMENT OF COMPUTER ENGINEERING

NBN SINHGAD SCHOOL OF ENGINEERING, PUNE

SAVITRIBAI PHULE PUNE UNIVERSITY, PUNE

ACADEMIC YEAR 2026-2027

Prof. Guide Name	Dr. S. P. Bendale
Internal Guide	H.O.D
Dept. of Computer Engg.	Dept. of Computer Engg.

Abstract

Web browsers record every page a user opens, every search they make and every file they download. In a digital-forensic investigation this browsing history is one of the richest sources of evidence about a suspect's activity, intent and timeline. Modern Chromium-based browsers (Google Chrome, Microsoft Edge, Brave) store this history in a single SQLite database called `History`, but the data is not directly readable: the file is locked while the browser runs, timestamps are stored as microseconds since 1 January 1601, and the information is spread across several related tables.

This mini project presents the design and development of a **Browser History Forensic Analysis Tool** that automates the standard forensic workflow of *acquire, preserve, analyse and report*. The tool copies the History database to a working location, computes and verifies a SHA-256 hash of the evidence so that its integrity can be demonstrated, parses the `urls`, `visits` and `downloads` tables, converts WebKit timestamps into human-readable local time, and produces a chronological timeline of browsing activity. It further summarises the most visited domains, daily activity, downloaded files and keyword hits, and exports the results as CSV files and a self-contained HTML report that can be attached to an investigation file.

The tool is written in about 280 lines of Python using only the standard library (`sqlite3`, `hashlib`, `csv`, `argparse`), so it runs on Windows, Linux and macOS without installing any third-party package. A sample database generator and a pytest suite of seven tests are included, and the tool was verified on a synthetic database and on the live Microsoft Edge history of a Windows 11 machine. The outcome is a small, transparent and easily explainable forensic utility that demonstrates the core principles of evidence handling, artifact parsing and timeline analysis taught in the Cyber Security and Digital Forensics course.

Acknowledgments

It gives us great pleasure in presenting the mini project report on ***'Browser History Forensic Analysis Tool'****.*

We would like to take this opportunity to thank our internal guide ***Prof. Guide Name*** *for giving us all the help and guidance we needed. We are really grateful to them for their kind support. Their valuable suggestions were very helpful.*

We are also grateful to ***Dr. S. P. Bendale****, Head of Computer Engineering Department, NBN Sinhgad School of Engineering, for his indispensable support and suggestions.*

In the end our special thanks to the ***laboratory staff of the Department of Computer Engineering*** *for providing various resources such as the laboratory with all needed software platforms and continuous Internet connection for our project.*

Vishwajeet Tulse
Student Name 2
Student Name 3
Student Name 4
(B.E. Computer Engg.)

Contents

1 Synopsis **1**
- 1.1 Project Title . . . 2
- 1.2 Project Option . . . 2
- 1.3 Internal Guide . . . 2
- 1.4 Sponsorship and External Guide . . . 2
- 1.5 Technical Keywords (As per ACM Keywords) . . . 2
- 1.6 Problem Statement . . . 2
- 1.7 Abstract . . . 3
- 1.8 Goals and Objectives . . . 3
- 1.9 Relevant mathematics associated with the Project . . . 3
- 1.10 Names of Conferences / Journals where papers can be published . . . 4
- 1.11 Review of Conference/Journal Papers supporting Project idea . . . 4
- 1.12 Plan of Project Execution . . . 5

2 Technical Keywords **6**
- 2.1 Area of Project . . . 7
- 2.2 Technical Keywords . . . 7

3 Introduction **8**
- 3.1 Project Idea . . . 9
- 3.2 Motivation of the Project . . . 9
- 3.3 Literature Survey . . . 9

4 Problem Definition and scope **11**
- 4.1 Problem Statement . . . 12
  - 4.1.1 Goals and objectives . . . 12
  - 4.1.2 Statement of scope . . . 12
- 4.2 Major Constraints . . . 12
- 4.3 Methodologies of Problem solving and efficiency issues . . . 13
- 4.4 Outcome . . . 13
- 4.5 Applications . . . 13
- 4.6 Hardware Resources Required . . . 13
- 4.7 Software Resources Required . . . 13

5 Project Plan **15**
- 5.1 Project Estimates . . . 16
  - 5.1.1 Reconciled Estimates . . . 16
  - 5.1.2 Project Resources . . . 16
- 5.2 Risk Management w.r.t. NP Hard analysis . . . 16

5.2.1 Risk Identification . . . 16
5.2.2 Risk Analysis . . . 17
5.2.3 Overview of Risk Mitigation, Monitoring, Management . . . 17
5.3 Project Schedule . . . 17
5.3.1 Project task set . . . 17
5.3.2 Task network . . . 19
5.3.3 Timeline Chart . . . 19
5.4 Team Organization . . . 20
5.4.1 Team structure . . . 20
5.4.2 Management reporting and communication . . . 20

6 Software requirement specification 21
6.1 Introduction . . . 22
6.1.1 Purpose and Scope of Document . . . 22
6.1.2 Overview of responsibilities of Developer . . . 22
6.2 Usage Scenario . . . 22
6.2.1 User profiles . . . 22
6.2.2 Use-cases . . . 22
6.2.3 Use Case View . . . 23
6.3 Data Model and Description . . . 23
6.3.1 Data Description . . . 23
6.3.2 Data objects and Relationships . . . 24
6.4 Functional Model and Description . . . 24
6.4.1 Data Flow Diagram . . . 24
6.4.2 Activity Diagram: . . . 26
6.4.3 Non Functional Requirements: . . . 26
6.4.4 State Diagram: . . . 27
6.4.5 Design Constraints . . . 27
6.4.6 Software Interface Description . . . 27

7 Detailed Design Document using Appendix A and B 28
7.1 Introduction . . . 29
7.2 Architectural Design . . . 29
7.3 Data design (using Appendices A and B) . . . 29
7.3.1 Internal software data structure . . . 29
7.3.2 Global data structure . . . 29
7.3.3 Temporary data structure . . . 29
7.3.4 Database description . . . 30
7.4 Component Design . . . 30
7.4.1 Class Diagram . . . 30

8 Project Implementation 31
8.1 Introduction . . . 32
8.2 Tools and Technologies Used . . . 32
8.3 Methodologies/Algorithm Details . . . 32
8.3.1 Algorithm 1/Pseudo Code . . . 32
8.3.2 Algorithm 2/Pseudo Code . . . 33
8.4 Verification and Validation for Acceptance . . . 33

9 Software Testing **34**
9.1 Type of Testing Used 35
9.2 Test Cases and Test Results 35

10 Results **36**
10.1 Screen shots 37
10.2 Outputs 38

11 Deployment and Maintenance **39**
11.1 Installation and un-installation 40
11.2 User help 40

12 Conclusion and Future Scope **41**

13 References **43**

Annexure A Laboratory assignments on Project Analysis of Algorithmic Design **45**

Annexure B Laboratory assignments on Project Quality and Reliability Testing of Project Design **47**

Annexure C Project Planner **49**

Annexure D Reviewers Comments of Paper Submitted **51**

Annexure E Plagiarism Report **53**

Annexure F Term-II Project Laboratory Assignments **55**

Annexure G Information of Project Group Members **57**

List of Figures

5.1 Project timeline (Gantt chart) 19

6.1 Use case diagram 23
6.2 Entity relationship of the History tables 24
6.3 Level 0 DFD 24
6.4 Level 1 DFD 25
6.5 Activity diagram 26
6.6 State transition diagram 27

7.1 Architecture diagram 29
7.2 Class / component diagram 30

10.1 Generated HTML forensic report (sample database, keyword “bank”) . . 37

List of Tables

4.1 Hardware Requirements 14

5.1 Time estimates 16
5.2 Risk Table 17
5.3 Risk Probability definitions [11] 17
5.4 Risk Impact definitions [11] 17

6.1 Use Cases 22
6.2 Chromium History tables used by the tool 23

9.1 Test cases and results 35

C.1 Project planner 50

CHAPTER 1
SYNOPSIS

1.1 Project Title

Design and Development of a Browser History Forensic Analysis Tool

1.2 Project Option

Internal project (mini project for Laboratory Practice IV, Cyber Security and Digital Forensics).

1.3 Internal Guide

Prof. Guide Name

1.4 Sponsorship and External Guide

Not applicable. The project is not sponsored.

1.5 Technical Keywords (As per ACM Keywords)

1. Security and privacy
    (a) Security services
        i. Digital forensics
        ii. Data integrity (cryptographic hashing)
2. Information systems
    (a) Data management systems
        i. Relational database query languages (SQLite / SQL)
3. Software and its engineering
    (a) Software notations and tools
        i. Command line tools, scripting languages (Python)

1.6 Problem Statement

Design and develop a digital-forensic tool that acquires the browsing-history database of a Chromium-based web browser (Chrome, Edge, Brave), preserves its integrity using cryptographic hashing, parses the visited URLs, page titles, timestamps and downloads, and generates a chronological timeline with summary statistics, keyword search and an exportable report for use in a forensic investigation.

1.7 Abstract

- Browser history is a key artifact in cyber-crime investigations because it reveals what a user searched for, which sites were visited, when, and what was downloaded.
- Chromium browsers keep this data in a locked SQLite file with non-standard timestamps, so it cannot be examined directly or safely.
- The proposed tool copies the evidence, verifies it with SHA-256, parses it with the Python `sqlite3` module, converts WebKit time to local time and builds a timeline.
- It reports the top visited domains, daily activity, downloads and keyword hits, and exports CSV and HTML reports.
- The implementation is a single Python file with no third-party dependencies, accompanied by a sample-database generator and automated tests.

1.8 Goals and Objectives

- To understand how Chromium-based browsers store browsing history and downloads.
- To acquire the History database without altering the original evidence and to prove integrity using SHA-256.
- To convert WebKit/Chrome timestamps into readable local date-time values.
- To reconstruct a chronological timeline of user activity.
- To provide analysis features useful to an investigator: top domains, daily activity, keyword search and download listing.
- To generate CSV and HTML reports that can be attached to a case file.
- To keep the tool small, portable and easy to explain.

1.9 Relevant mathematics associated with the Project

System Description:

- **Input:** H = the browser History SQLite file; k = optional keyword; n = number of top domains.
- **Output:** evidence copy H' with hash $h = \mathrm{SHA256}(H)$; ordered timeline $T = \langle t_1, t_2, \ldots, t_m \rangle$; domain frequency table D; keyword hit set K; downloads list L; files `timeline.csv`, `downloads.csv`, `report.html`.
- **Data structures:** list of visit dictionaries (time, url, title, domain, visit_count, typed_count); `Counter` (hash map) for domain and daily frequencies; SQL join of relations `visits` $\bowtie$ `urls`.

- **Functions:**
  - $f_{acq} : H \to (H', h)$ with the success condition SHA256(H) = SHA256(H').
  - $f_{time} : \mu \to t$, where $t = \mu/10^6 - 11644473600$ seconds since the Unix epoch (μ = microseconds since 1601-01-01).
  - $f_{parse} : H' \to T$, T sorted by visit time.
  - $f_{top} : (T, n) \to D$, the n most frequent domains.
  - $f_{key} : (T, k) \to K = \{v \in T \mid k \subseteq \text{lower}(v.url) \vee k \subseteq \text{lower}(v.title)\}$.
- **Mathematical formulation:** the timeline is a totally ordered set under visit time; domain ranking is a frequency count $D(d) = |\{v \in T : \text{domain}(v) = d\}|$. Complexity is $O(m \log m)$ for sorting (done by SQLite) and $O(m)$ for every analysis pass.
- **Success Conditions:** History file is found and readable; hash of copy equals hash of source; timeline and reports are produced.
- **Failure Conditions:** file not found; file is not a valid SQLite database or lacks the expected tables; hash mismatch after copy (tool aborts to protect integrity).

1.10 Names of Conferences / Journals where papers can be published

- IEEE International Conference on Computing, Communication and Networking Technologies (ICCCNT)
- Digital Investigation / Forensic Science International: Digital Investigation (Elsevier)
- International Conference on Digital Forensics and Cyber Crime (ICDF2C)
- SPPU-sponsored and IIT-hosted student technical conferences / workshops

1.11 Review of Conference/Journal Papers supporting Project idea

A brief review of the literature consulted is given below; the detailed survey is in Chapter 3.

1. Oh, Lee and Lee [1] describe browser log analysis as a central step in digital investigations and propose a timeline-based methodology, which this project follows.
2. Ohana and Shashidhar [2] show that private-browsing and portable browsers still leave recoverable artifacts, motivating careful acquisition.
3. Rathod [3] documents the Chrome artifacts (History, Cookies, Login Data) and their SQLite structure.

4. Casey [4] and Carrier [5] give the general principles of evidence integrity and file-system forensics used for the acquisition step.

5. The Chromium project documentation [8] and the SQLite documentation [9] define the schema and the file format parsed by the tool.

6. Kessler [7] explains cryptographic hashing (MD5/SHA) as the standard integrity mechanism in forensics.

7. NIST SP 800-86 [6] provides the four-phase model (collection, examination, analysis, reporting) adopted as the tool's architecture.

8. Pressman [11] is used for the project-planning and risk-analysis framework.

1.12 Plan of Project Execution

The project was executed in five short phases over four weeks: (1) study of Chromium History schema and forensic principles, (2) design of the acquire-parse-analyse-report pipeline, (3) implementation in Python, (4) testing with a synthetic database and a live Edge profile, (5) report preparation. See Annexure C for the planner.

CHAPTER 2
TECHNICAL KEYWORDS

2.1 Area of Project

Cyber Security and Digital Forensics: browser artifact analysis, evidence acquisition and integrity verification.

2.2 Technical Keywords

1. Security and privacy
   (a) Digital forensics
   (b) Hash functions and message authentication codes
2. Information systems
   (a) Structured Query Language, relational databases (SQLite)
3. Applied computing
   (a) Computer forensics, evidence collection, timeline analysis
4. Software and its engineering
   (a) Scripting languages (Python), command-line tools, software testing (pytest)

CHAPTER 3
INTRODUCTION

3.1 Project Idea

- Every Chromium-based browser keeps a per-profile SQLite file named `History` containing the tables `urls` (one row per unique URL with title and visit counts), `visits` (one row per visit with a timestamp) and `downloads`.
- The idea is to build a small forensic tool that reads this file in a forensically sound way and turns the raw rows into information an investigator can use: a timeline, frequency statistics, keyword hits and a printable report.
- The tool is deliberately kept simple: one Python file, standard library only, command-line interface, so that each step can be explained and verified.

3.2 Motivation of the Project

- Commercial forensic suites (EnCase, FTK, Magnet AXIOM) are expensive and act as black boxes; students rarely get to see how an artifact is actually parsed.
- The LP-IV syllabus asks for a tool for digital forensics of a user artifact; browser history is universal, easily available and highly relevant to real cases (phishing, fraud, data exfiltration, insider threats).
- Building the parser by hand teaches three essential forensic concepts at once: evidence integrity (hashing), artifact structure (SQLite schema) and timeline reconstruction (timestamp conversion and ordering).

3.3 Literature Survey

- **Oh, Lee and Lee (2011)** [1] proposed an advanced methodology for web-browser log analysis that integrates history, cache and cookies into a single timeline and highlights searched keywords. Our tool implements the history-and-download part of this methodology.
- **Ohana and Shashidhar (2013)** [2] examined private and portable browsing sessions and found that significant artifacts remain on disk, confirming that the History database is a reliable evidence source.
- **Rathod (2017)** [3] analysed Google Chrome's forensic artifacts on Windows, listing the History, Cookies, Login Data and Top Sites SQLite files and their key columns; the `urls`/`visits` join used here follows that description.
- **Casey (2011)** [4] sets out the principles of digital evidence handling, including working on a verified copy rather than the original.
- **Carrier (2005)** [5] explains file-system level analysis and the role of hashing in proving that an image has not changed.
- **NIST SP 800-86 (2006)** [6] defines the collection, examination, analysis and reporting phases; the tool's four steps map directly to these phases.

- **Kessler (2016)** [7] gives an overview of cryptographic hash functions and why SHA-256 is preferred over MD5 for evidence verification today.
- **Chromium design documentation** [8] documents the WebKit time format: a 64-bit count of microseconds since 1 January 1601 UTC, which must be shifted by 11 644 473 600 seconds to obtain Unix time.
- **SQLite documentation** [9] describes the single-file database format and the read-only URI mode used to open the evidence copy without modification.
- **Python documentation** [10] for the `sqlite3`, `hashlib` and `datetime` modules used in the implementation.

CHAPTER 4
PROBLEM DEFINITION AND SCOPE

4.1 Problem Statement

Investigators need to know what a user did in a web browser and when. The required information exists in the browser's History database, but it is locked by the running browser, uses a non-standard timestamp format and is spread over several tables. Reading it manually is slow and error-prone, and opening the original file risks modifying the evidence. The problem is therefore to build a tool that *safely acquires* the History file, *verifies* its integrity, *parses* it into a readable timeline and *reports* the findings.

4.1.1 Goals and objectives

- Input: path of a History file, or the name of a browser (`chrome`, `edge`, `brave`) from which the default path is derived; optional keyword and top-n value.
- Output: an evidence copy with its SHA-256 digest, `timeline.csv`, `downloads.csv` and `report.html`, plus a console summary.
- Objective: correctness of timestamps, completeness of the timeline, preservation of the original, and a clear report.

4.1.2 Statement of scope

- The tool handles Chromium-family browsers (Chrome, Edge, Brave) on Windows, Linux and macOS. The History file is typically a few hundred kilobytes to a few hundred megabytes; the tool reads it in a single pass.
- Input validation: the path must exist and must be a SQLite database with `urls` and `visits` tables; the `downloads` table is optional.
- The tool *does* produce timelines, domain statistics, daily activity, keyword hits and download lists.
- The tool *does not* parse Firefox (`places.sqlite`), cookies, cache, saved passwords or deleted (free-page) records, and it does not modify the source file.

4.2 Major Constraints

- The original History file must never be written to; the tool opens only the copy, in read-only mode.
- Only the Python standard library may be used, so the tool runs on an examiner workstation without Internet access.
- Browser History schema may change between versions; the tool uses only long-stable columns (`url`, `title`, `visit_time`, `visit_count`, `typed_count`, `target_path`, `tab_url`, `start_time`).
- Reading a live user profile requires that the examiner has file-system access to the profile folder.

4.3 Methodologies of Problem solving and efficiency issues

- **Manual inspection with a SQLite GUI:** possible but slow, offers no hashing, no timestamp conversion and no reporting.
- **Commercial forensic suite:** complete but costly and opaque; unsuitable for a learning project.
- **Chosen approach, custom Python script:** a single SQL join retrieves all visits ordered by time ($O(m \log m)$ inside SQLite), and each analysis pass is linear in the number of visits. A 100 000-visit database is processed in well under a second; the HTML report is generated with string concatenation, which is adequate for this size.

4.4 Outcome

- A working command-line forensic tool (`bhft.py`) with hashing, parsing, analysis and reporting.
- A synthetic History generator for safe demonstrations and a pytest suite that verifies every function.
- A documented GitHub repository and this report.

4.5 Applications

- Cyber-crime investigation: establishing which websites a suspect visited and when.
- Corporate incident response: checking whether a phishing link was opened or a malicious file downloaded.
- Insider-threat and data-leak investigations: identifying uploads to file-sharing or webmail sites.
- Parental or institutional policy audits, and teaching of digital forensics.

4.6 Hardware Resources Required

4.7 Software Resources Required

Platform :

1. Operating System: Windows 10/11, Linux or macOS
2. IDE: Visual Studio Code (any text editor is sufficient)
3. Programming Language: Python 3.9 or newer (standard library only); pytest for the test suite

Sr. No.	Parameter	Minimum Requirement	Justification
1	CPU Speed	1 GHz	Parsing is linear; any modern CPU suffices
2	RAM	2 GB	The whole History table is held in memory; typical files are well under 100 MB
3	Disk	500 MB free	Space for the evidence copy and reports

Table 4.1: Hardware Requirements

4. Browser for viewing the HTML report; Git and GitHub for version control

CHAPTER 5
PROJECT PLAN

5.1 Project Estimates

The project followed a simple waterfall model: requirements, design, implementation, testing and documentation.

5.1.1 Reconciled Estimates

5.1.1.1 Cost Estimate

All software used is free and open source (Python, pytest, Git, VS Code, MiKTeX). The only cost is student effort, estimated at 40 person-hours for a team of four, and use of existing laboratory computers.

5.1.1.2 Time Estimates

Phase	Duration
Study of History schema and forensic principles	4 days
Design (pipeline, data structures, CLI)	2 days
Implementation of `bhft.py` and sample generator	5 days
Testing (pytest, live Edge profile)	3 days
Report and repository documentation	4 days
Total	**18 working days**

Table 5.1: Time estimates

5.1.2 Project Resources

People: four students and one guide. Hardware: one laptop per student. Software: Python 3, pytest, Git/GitHub, VS Code, Microsoft Edge (for a live test profile), MiKTeX for the report.

5.2 Risk Management w.r.t. NP Hard analysis

The problem is not NP-hard: every step is a linear or $O(m \log m)$ pass over the visit rows. Risks are therefore practical rather than computational.

5.2.1 Risk Identification

Risks were identified by reviewing the scope and the environment in which the tool runs, using the questionnaire suggested in [11]. The relevant answers: requirements are stable and fully understood; the team has the required Python skills; end-user expectations (a simple timeline tool) are realistic.

5.2.2 Risk Analysis

ID	Risk Description	Probability	Impact		
			Schedule	Quality	Overall
1	Browser History schema changes in a new version	Low	Low	High	Medium
2	Locked History file cannot be copied while browser runs	Medium	Low	Medium	Medium
3	Wrong timestamp conversion produces incorrect timeline	Low	Low	Very high	High

Table 5.2: Risk Table

Probability	Value	Description
High	Probability of occurrence is	$> 75\%$
Medium	Probability of occurrence is	$26 - 75\%$
Low	Probability of occurrence is	$< 25\%$

Table 5.3: Risk Probability definitions [11]

Impact	Value	Description
Very high	$> 10\%$	Schedule impact or Unacceptable quality
High	$5 - 10\%$	Schedule impact or Some parts of the project have low quality
Medium	$< 5\%$	Schedule impact or Barely noticeable degradation in quality

Table 5.4: Risk Impact definitions [11]

5.2.3 Overview of Risk Mitigation, Monitoring, Management

5.3 Project Schedule

5.3.1 Project task set

Major Tasks in the Project stages are:

- Task 1: Study Chromium History schema, WebKit time format and forensic acquisition principles.

Risk ID	1
Risk Description	Browser History schema changes in a new version
Category	Technology
Source	Study of Chromium source and release notes
Probability	Low
Impact	High
Response	Mitigate
Strategy	Use only long-stable columns; wrap the optional `downloads` query in a try/except so the tool degrades gracefully
Risk Status	Mitigated

Risk ID	2
Risk Description	Locked History file cannot be read while the browser runs
Category	Development Environment
Source	Observed during testing on a live Edge profile
Probability	Medium
Impact	Medium
Response	Mitigate
Strategy	Copy the file with `shutil.copy2` (a byte copy is allowed even when SQLite holds a lock) and open only the copy in read-only mode
Risk Status	Mitigated

Risk ID	3
Risk Description	Wrong timestamp conversion
Category	Requirements / Technology
Source	Chromium documentation review
Probability	Low
Impact	Very High
Response	Mitigate
Strategy	Unit test that converts the known WebKit value of the Unix epoch and checks the result is 1970-01-01
Risk Status	Mitigated (test passes)

- Task 2: Design the four-step pipeline and the command-line interface.
- Task 3: Implement acquisition, parsing, analysis and reporting functions.
- Task 4: Build the sample-database generator and the pytest suite; test on a live profile.
- Task 5: Prepare the GitHub repository and this report.

5.3.2 Task network

Task 1 → Task 2 → Task 3 → Task 4 → Task 5. Task 4 (sample generator) was started in parallel with Task 3 because the tests needed a database before the parser was complete.

5.3.3 Timeline Chart

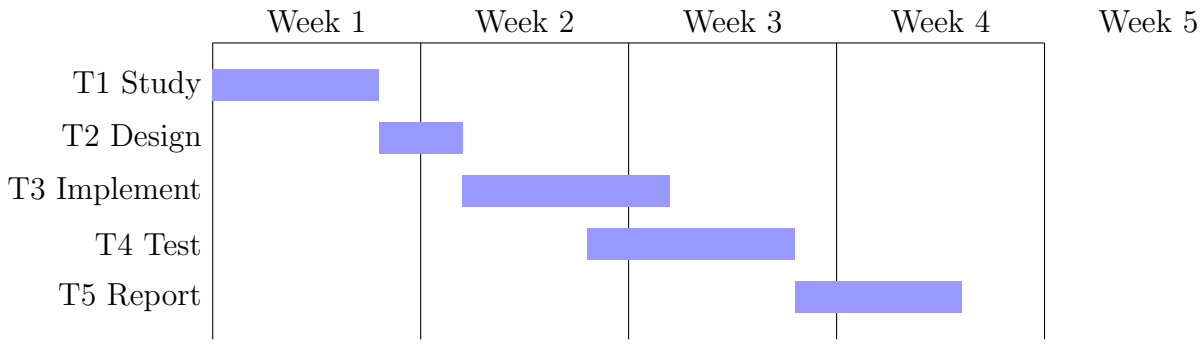


Figure 5.1: Project timeline (Gantt chart)

Please refer to Annexure C for the planner.

5.4 Team Organization

5.4.1 Team structure

A flat team of four students with a shared repository. Roles: (1) parsing and timestamp logic, (2) acquisition, hashing and CLI, (3) reporting (CSV/HTML) and sample generator, (4) testing and documentation. All members reviewed each other's code through GitHub commits.

5.4.2 Management reporting and communication

Weekly progress was shown to the guide during the LP-IV laboratory slot. Day-to-day communication used a group chat, and all work was tracked through commits in the GitHub repository.

CHAPTER 6

SOFTWARE REQUIREMENT SPECIFICATION

6.1 Introduction

6.1.1 Purpose and Scope of Document

This SRS describes the functional and non-functional requirements of the Browser History Forensic Analysis Tool so that it can be implemented, tested and evaluated. It covers acquisition, parsing, analysis and reporting of Chromium browser history.

6.1.2 Overview of responsibilities of Developer

The developers study the History schema, implement the tool, write a sample-database generator and automated tests, verify the tool on real data, and document the result.

6.2 Usage Scenario

6.2.1 User profiles

- **Forensic examiner / investigator:** runs the tool on a seized History file or on a live profile and reads the report.
- **Student / learner:** runs the tool on the sample database to understand browser artifacts.

6.2.2 Use-cases

Sr No.	Use Case	Description	Actors	Assumptions
1	Acquire evidence	Copy History file and compute SHA-256	Examiner	Examiner has read access to the profile folder
2	Build timeline	Parse visits and show them in time order	Examiner	File is a valid Chromium History DB
3	Search keyword	List visits whose URL/title contains a word	Examiner	Keyword given on the command line
4	View statistics	Top domains and daily activity counts	Examiner	At least one visit exists
5	List downloads	Show downloaded files with source URL	Examiner	`downloads` table present
6	Export report	Write CSV and HTML report	Examiner	Output folder is writable

Table 6.1: Use Cases

6.2.3 Use Case View

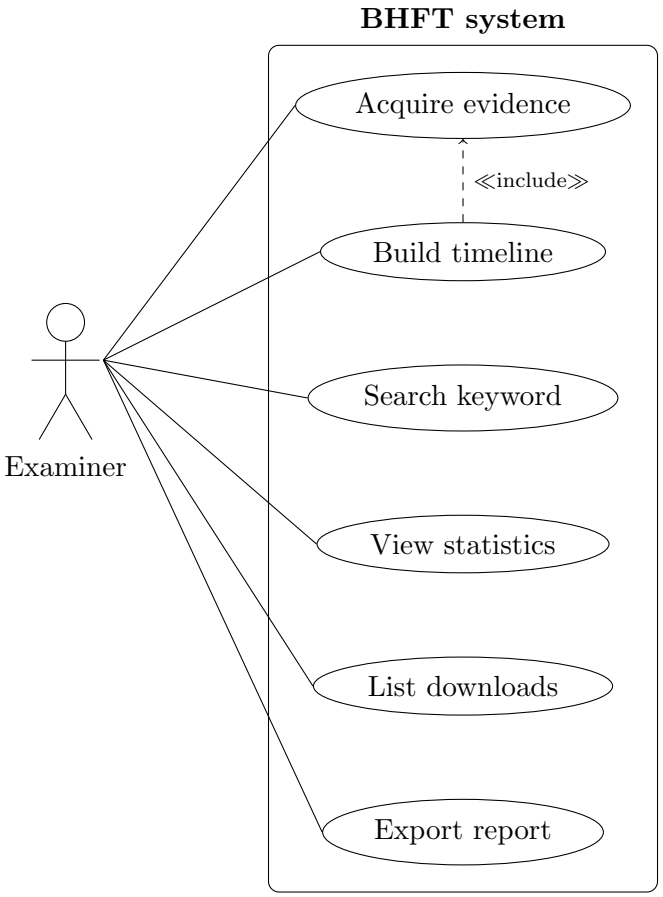


Figure 6.1: Use case diagram

6.3 Data Model and Description

6.3.1 Data Description

The tool reads three tables of the Chromium History database (Table 6.2) and produces a list of *visit records* and *download records* in memory, which are then written to CSV and HTML.

Table	Column	Meaning
`urls`	id, url, title, visit_count, typed_count, last_visit_time	one row per unique URL
`visits`	id, url (FK to urls.id), visit_time, from_visit, transition	one row per visit
`downloads`	id, target_path, tab_url, start_time, received_bytes	one row per download

Table 6.2: Chromium History tables used by the tool

6.3.2 Data objects and Relationships

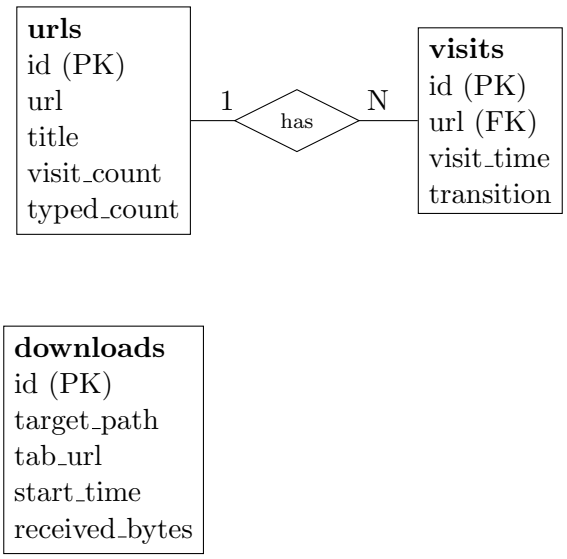


Figure 6.2: Entity relationship of the History tables

6.4 Functional Model and Description

6.4.1 Data Flow Diagram

6.4.1.1 Level 0 Data Flow Diagram

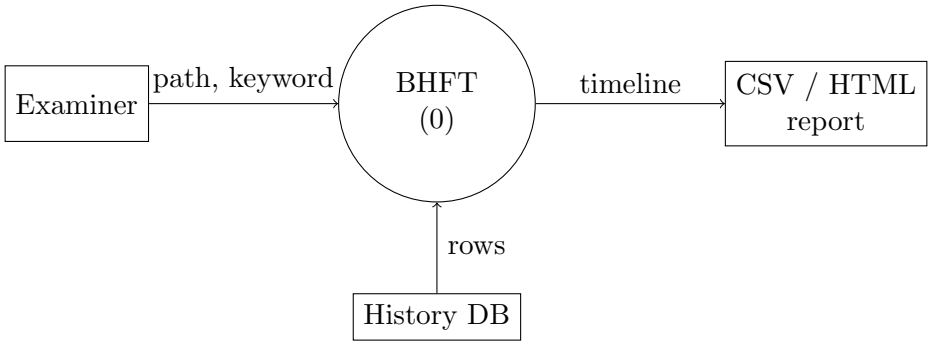


Figure 6.3: Level 0 DFD

6.4.1.2 Level 1 Data Flow Diagram

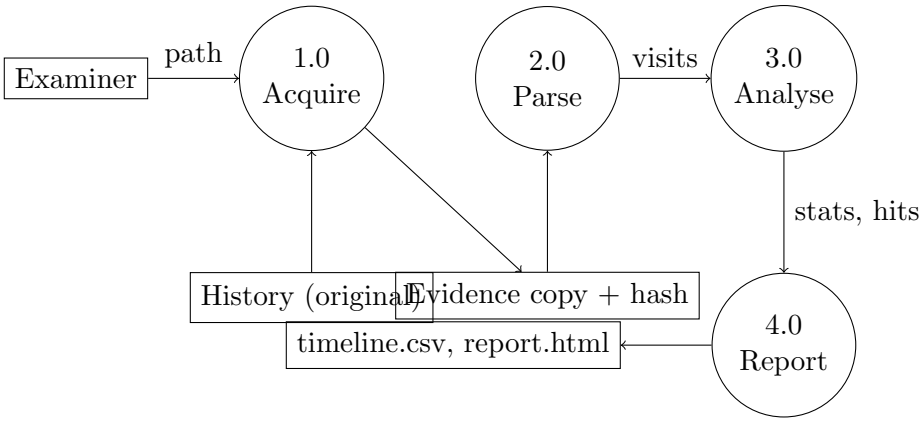


Figure 6.4: Level 1 DFD

6.4.2 Activity Diagram:

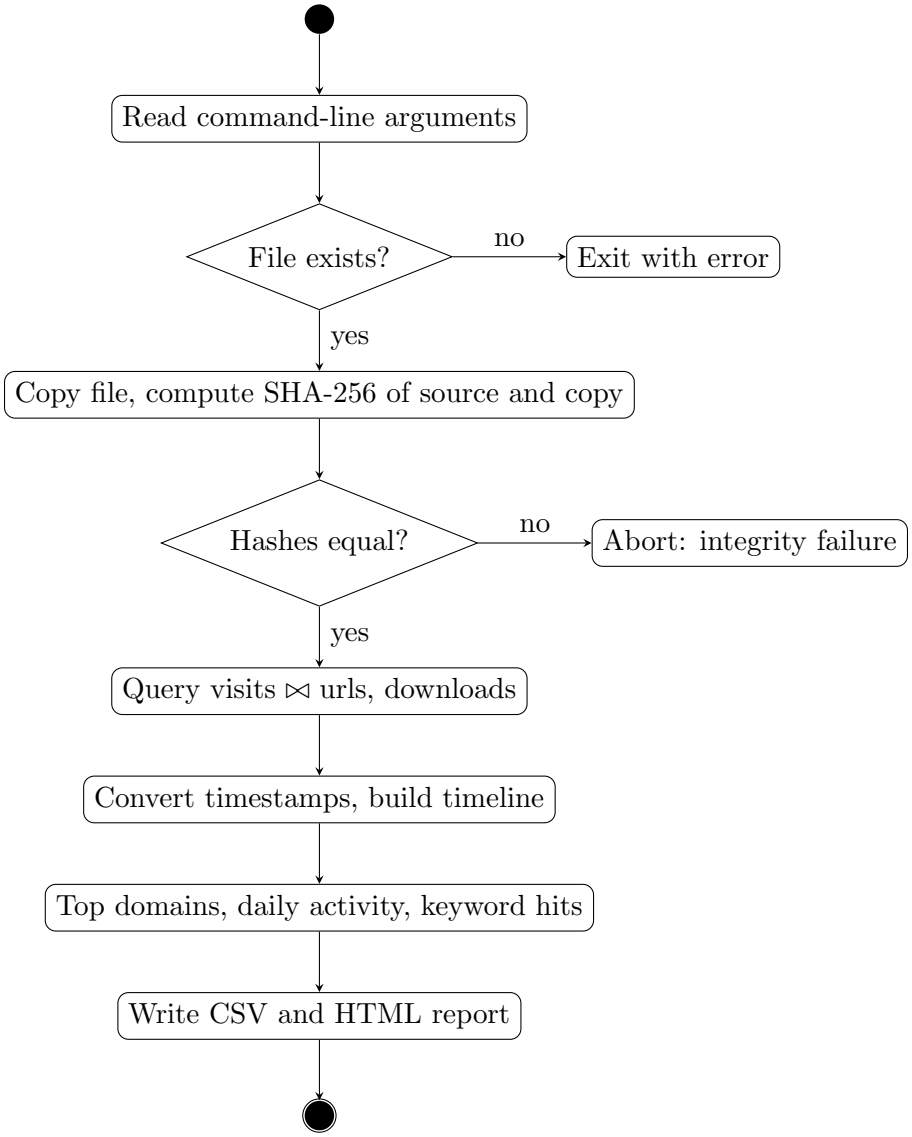


Figure 6.5: Activity diagram

6.4.3 Non Functional Requirements:

- **Interface Requirements:** command-line interface with `--browser`, `--db`, `--out`, `--keyword`, `--top`; HTML report viewable in any browser.
- **Performance Requirements:** process a 100 000-visit database in under 2 seconds on the minimum hardware.
- **Availability / Reliability:** no network dependency; deterministic output for the same input.
- **Modifiability / Portability:** single file, standard library only, runs on Windows, Linux and macOS.
- **Security:** the source file is opened read-only; the evidence copy is hash-verified.

- **Testability:** every function is unit-tested with pytest; an end-to-end test runs the CLI.
- **Usability:** one command produces all outputs; the report is self-explanatory.

6.4.4 State Diagram:

Fig.6.6 shows the states of the tool during one run.

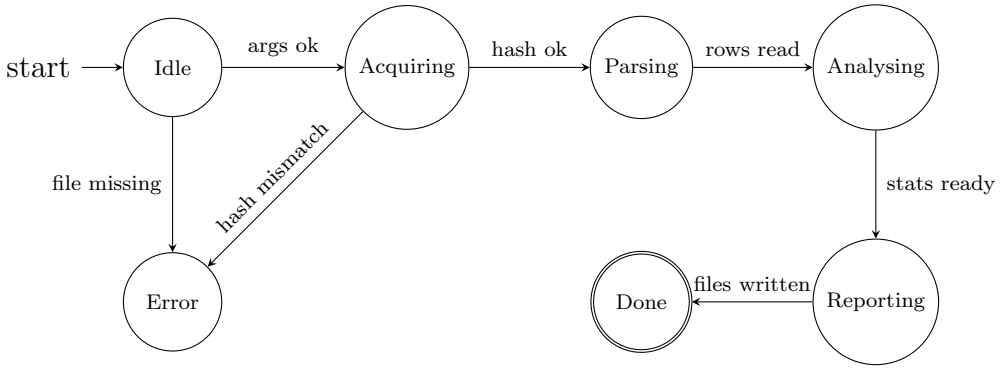


Figure 6.6: State transition diagram

6.4.5 Design Constraints

Standard library only; no modification of the source file; output must be plain CSV and HTML so that it can be opened on any machine.

6.4.6 Software Interface Description

- **Input interface:** file system (History file) and command-line arguments.
- **Database interface:** Python `sqlite3` module, opened with the URI `file:...?mode=ro`.
- **Output interface:** files in the output folder; console summary on standard output.

CHAPTER 7

DETAILED DESIGN DOCUMENT USING APPENDIX A AND B

7.1 Introduction

This chapter describes the architecture, data design and component design used to implement the tool.

7.2 Architectural Design

The tool is a linear pipeline of four modules inside one Python file (Fig.7.1), mirroring the NIST forensic process.

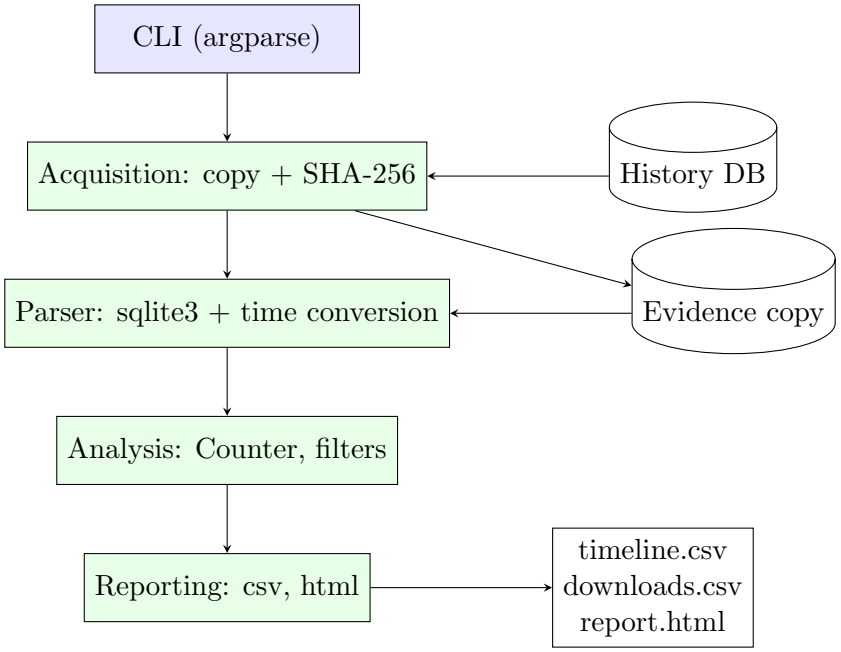


Figure 7.1: Architecture diagram

7.3 Data design (using Appendices A and B)

7.3.1 Internal software data structure

A *visit record* is a Python dictionary with keys `time`, `url`, `title`, `domain`, `visit_count`, `typed_count`. A *download record* has `time`, `file`, `url`, `bytes`. Both are passed between the parse, analyse and report functions as lists.

7.3.2 Global data structure

`WEBKIT_EPOCH_DELTA = 11644473600`, the constant used for timestamp conversion, and the dictionary of default profile paths per browser and operating system.

7.3.3 Temporary data structure

`collections.Counter` objects for domain and daily frequencies; the list of HTML fragments assembled before writing the report.

7.3.4 Database description

The input is the Chromium History SQLite database (Table 6.2). The tool creates no database of its own; the evidence copy `History_evidence_copy.db` is a byte-for-byte copy of the input.

7.4 Component Design

7.4.1 Class Diagram

The implementation is procedural, but its functions group naturally into four components shown as classes in Fig.7.2.

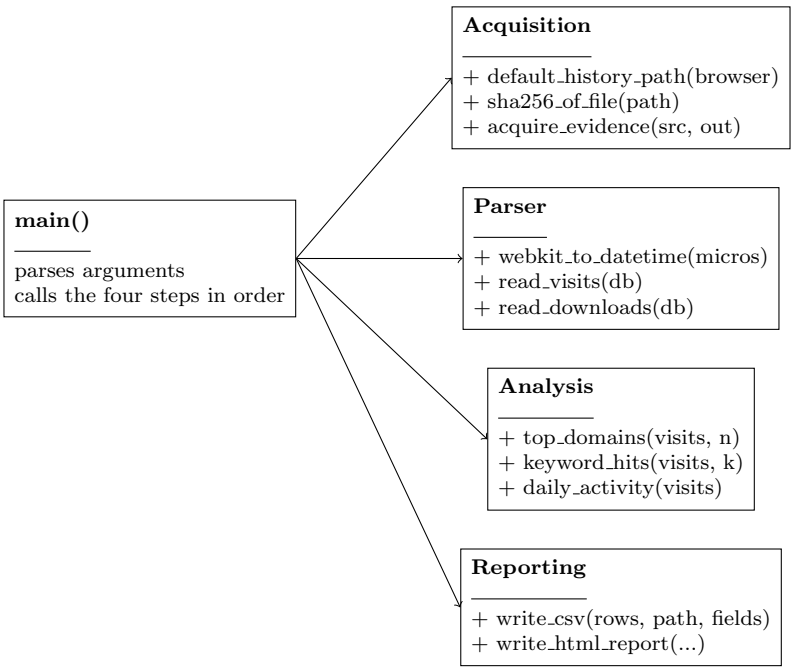


Figure 7.2: Class / component diagram

CHAPTER 8
PROJECT IMPLEMENTATION

8.1 Introduction

The tool was implemented as a single script `bhft.py` (about 280 lines), with a helper script `make_sample_db.py` that generates a realistic synthetic History database and a test module `test_bhft.py`. The code is hosted at `https://github.com/VishwajeetTulse/browser-history-forensic-tool`.

8.2 Tools and Technologies Used

- Python 3.14 (standard library: `sqlite3`, `hashlib`, `csv`, `argparse`, `datetime`, `shutil`, `html`, `collections`)
- pytest 9 for automated tests
- Git and GitHub for version control
- Visual Studio Code; Microsoft Edge (live test profile and report viewing)

8.3 Methodologies/Algorithm Details

8.3.1 Algorithm 1/Pseudo Code

Algorithm 1: Evidence acquisition with integrity check

Input: source path *src*, output folder *out*
Output: copy path *dst*, digest h
create *out* if missing;
$dst \leftarrow out$/`History_evidence_copy.db`;
byte-copy *src* to *dst*;
$h_s \leftarrow \text{SHA256}(src)$; $h_d \leftarrow \text{SHA256}(dst)$;
if $h_s \neq h_d$ **then**
    abort with "integrity cannot be guaranteed";
return (dst, h_s);

8.3.2 Algorithm 2/Pseudo Code

Algorithm 2: Timeline construction and analysis

Input: evidence copy *dst*, keyword k, count n
Output: timeline T, top domains D, hits K, daily counts A
open *dst* read-only;
rows ← `SELECT v.visit_time, u.url, u.title, u.visit_count, u.typed_count FROM visits v JOIN urls u ON v.url=u.id ORDER BY v.visit_time`;
foreach *row in rows* **do**
  $t \leftarrow$ row.visit_time/$10^6 - 11644473600$ seconds since 1970, converted to local time;
  append (t, url, title, domain(url), counts) to T;
$D \leftarrow n$ most common domains in T;
$K \leftarrow \{v \in T : k \in \text{lower}(v.url) \vee k \in \text{lower}(v.title)\}$;
$A \leftarrow$ count of visits per calendar date;
return (T, D, K, A);

Key implementation of the timestamp conversion:

```
WEBKIT_EPOCH_DELTA = 11644473600  # seconds between 1601-01-01 and 1970-01-01

def webkit_to_datetime(micros):
    """Chromium stores times as microseconds since 1601-01-01 UTC."""
    if not micros:
        return None
    seconds = micros / 1_000_000 - WEBKIT_EPOCH_DELTA
    return dt.datetime.fromtimestamp(seconds, dt.timezone.utc).astimezone()
```

8.4 Verification and Validation for Acceptance

Acceptance criteria: (a) hash of copy equals hash of source, (b) timeline is in increasing time order, (c) all visits in the database appear in the timeline, (d) keyword search is case-insensitive, (e) CSV and HTML files are produced. All criteria are checked automatically by the pytest suite (Chapter 9) and were also confirmed manually on a live Microsoft Edge profile.

CHAPTER 9
SOFTWARE TESTING

9.1 Type of Testing Used

- **Unit testing** of individual functions (timestamp conversion, hashing, parsing, analysis) with pytest.
- **Integration / end-to-end testing** by running the complete CLI on the sample database and checking the output files.
- **System testing** by running the tool against the live Microsoft Edge profile of a Windows 11 machine.

9.2 Test Cases and Test Results

TC	Test description	Expected result	Result
TC1	Convert WebKit value of Unix epoch	1970-01-01; value 0 gives None	Pass
TC2	SHA-256 of sample file	equals hashlib digest	Pass
TC3	Read visits from sample DB	non-empty, sorted by time, every row has a domain	Pass
TC4	Read downloads	3 rows, first is a .pdf	Pass
TC5	Keyword search "BANK" vs "bank"	same hits; unknown keyword gives empty list	Pass
TC6	Top-3 domains	3 entries in non-increasing order	Pass
TC7	End-to-end CLI run with `--keyword bank`	exit 0; report.html, timeline.csv, evidence copy exist; "SHA-256" printed	Pass
TC8	System test on live Edge profile	tool copies locked file, prints hash and timeline	Pass

Table 9.1: Test cases and results

pytest output:

```
test_bhft.py::test_webkit_epoch_conversion PASSED
test_bhft.py::test_sha256_matches_hashlib PASSED
test_bhft.py::test_read_visits_sorted_and_complete PASSED
test_bhft.py::test_downloads_read PASSED
test_bhft.py::test_keyword_search_case_insensitive PASSED
test_bhft.py::test_top_domains_order PASSED
test_bhft.py::test_end_to_end_cli PASSED
============================== 7 passed in 0.21s ==============================
```

CHAPTER 10

RESULTS

10.1 Screen shots



Browser History Forensic Analysis Report

1. Evidence Details

Source file	samples/History
SHA-256 of evidence	94f700d91be6a7f4e05f483ac4db8267bac466fbb7d7e25ccf56e52cbefdf9a2
Acquired on	2026-10-22 22:26:45
Examiner machine	DESKTOP-ML5U81T
Keyword	bank

2. Summary

Total visits	56
Unique URLs	14
Unique domains	13
Downloads	3
First visit	2026-10-13 09:38:00
Last visit	2026-10-17 17:11:00

3. Top Visited Domains

#	Domain	Visits
1	docs.python.org	7
2	www.wikipedia.org	7
3	www.youtube.com	7
4	netbanking.examplebank.com	6
5	www.sppu.in	5
6	www.amazon.in	4
7	www.linkedin.com	4
8	github.com	4
9	stackoverflow.com	3
10	drive.google.com	3

4. Daily Activity

Date	Visits
2026-10-13	8
2026-10-14	8
2026-10-15	12
2026-10-16	14
2026-10-17	14

5. Keyword Hits for 'bank' (6)

Time	Title	URL
2026-10-15 11:16:00	Example Bank - Net Banking Login	https://netbanking.examplebank.com/login
2026-10-15 12:02:00	Example Bank - Fund Transfer	https://netbanking.examplebank.com/transfer
2026-10-15 12:55:00	Example Bank - Net Banking Login	https://netbanking.examplebank.com/login
2026-10-16 10:47:00	Example Bank - Net Banking Login	https://netbanking.examplebank.com/login
2026-10-16 12:33:00	Example Bank - Fund Transfer	https://netbanking.examplebank.com/transfer
2026-10-17 10:51:00	Example Bank - Net Banking Login	https://netbanking.examplebank.com/login

6. Downloads

Time	File	Source URL	Bytes
2026-10-14 11:00:00	C:\Users\student\Downloads\sqlite3_manual.pdf	https://docs.python.org/3/library/sqlite3.html	482311

Figure 10.1: Generated HTML forensic report (sample database, keyword “bank”)

10.2 Outputs

Console output of `python bhft.py --db samples/History --out output --keyword bank`:

```
======================================================================
 Browser History Forensic Analysis Tool
======================================================================
[1] Acquiring evidence from: samples/History
    SHA-256 : 94f700d91be6a7f4e05f483ac4db8267bac466fbb7d7e25ccf56e52cbefdf9a2
    Working copy saved to: output\History_evidence_copy.db
[2] Parsing history database ...
    56 visits, 3 downloads found
[3] Analysing ...

    Top 10 domains:
          7  docs.python.org
          7  www.wikipedia.org
          7  www.youtube.com
          6  netbanking.examplebank.com
          5  www.sppu.in
          4  www.amazon.in
          4  www.linkedin.com
          4  github.com
          3  stackoverflow.com
          3  drive.google.com

    Activity window: 2026-10-13 09:38:00  ->  2026-10-17 17:11:00

    Keyword 'bank': 6 hit(s)
      2026-10-15 11:16:00  https://netbanking.examplebank.com/login
      2026-10-15 12:02:00  https://netbanking.examplebank.com/transfer
      2026-10-15 12:55:00  https://netbanking.examplebank.com/login
      2026-10-16 10:47:00  https://netbanking.examplebank.com/login
      2026-10-16 12:33:00  https://netbanking.examplebank.com/transfer
      2026-10-17 10:51:00  https://netbanking.examplebank.com/login

[4] Writing reports ...
    timeline.csv, downloads.csv and report.html written to 'output'
Done.
```

First rows of `timeline.csv`:

```
time,url,title,domain,visit_count,typed_count
2026-10-13 09:38:00,https://www.amazon.in/,Amazon.in,www.amazon.in,4,2
2026-10-13 09:44:00,https://www.linkedin.com/feed/,Feed | LinkedIn,www.linkedin.com
    ,4,0
```

CHAPTER 11
DEPLOYMENT AND MAINTENANCE

11.1 Installation and un-installation

1. Install Python 3.9 or newer.
2. Clone the repository from `https://github.com/VishwajeetTulse/browser-history-forensic`
3. (Optional) `pip install pytest` to run the tests.
4. Un-installation: delete the folder. The tool writes nothing outside its output directory.

11.2 User help

```
python make_sample_db.py                        # create demo database
python bhft.py --db samples/History --keyword bank
python bhft.py --browser edge                   # live Edge profile
python bhft.py --browser chrome --out case42 --top 20
python bhft.py --help                           # all options
```

Open `output/report.html` in a browser. Keep `History_evidence_copy.db` together with the printed SHA-256 value as part of the case record.

CHAPTER 12
CONCLUSION AND FUTURE SCOPE

The Browser History Forensic Analysis Tool demonstrates, in a small and transparent way, the complete forensic workflow for a common digital artifact. It acquires the Chromium History database without touching the original, proves integrity with SHA-256, decodes WebKit timestamps, reconstructs a chronological timeline and summarises the activity in a report that an investigator can use directly. All functions are covered by automated tests and the tool was validated on real browser data.

Future scope includes support for Firefox (`places.sqlite`) and Safari, parsing of cookies, cache and form data, recovery of deleted records from SQLite free pages, a graphical interface, and correlation of several users' timelines in one case.

CHAPTER 13
REFERENCES

[1] J. Oh, S. Lee and S. Lee. 2011. Advanced evidence collection and analysis of web browser activity. *Digital Investigation* 8 (2011), S62–S70.

[2] D. J. Ohana and N. Shashidhar. 2013. Do private and portable web browsers leave incriminating evidence? A forensic analysis of residual artifacts from private and portable web browsing sessions. *EURASIP Journal on Information Security* 2013, 6.

[3] D. Rathod. 2017. Web browser forensics: Google Chrome. *International Journal of Advanced Research in Computer Science* 8, 7 (2017), 896–899.

[4] E. Casey. 2011. *Digital Evidence and Computer Crime* (3rd ed.). Academic Press.

[5] B. Carrier. 2005. *File System Forensic Analysis*. Addison-Wesley.

[6] K. Kent, S. Chevalier, T. Grance and H. Dang. 2006. *Guide to Integrating Forensic Techniques into Incident Response*. NIST Special Publication 800-86.

[7] G. C. Kessler. 2016. An overview of cryptographic hash functions and their uses in digital forensics. *Journal of Digital Forensics, Security and Law*.

[8] The Chromium Projects. *User Data Directory and History database design*. `https://www.chromium.org/developers/design-documents/user-data-dir/`

[9] SQLite Consortium. *SQLite Documentation: Database File Format and URI Filenames*. `https://www.sqlite.org/docs.html`

[10] Python Software Foundation. *The Python Standard Library: sqlite3, hashlib, datetime*. `https://docs.python.org/3/library/`

[11] R. S. Pressman. 2010. *Software Engineering: A Practitioner's Approach* (7th ed.). McGraw-Hill.

ANNEXURE A

LABORATORY ASSIGNMENTS ON PROJECT ANALYSIS OF ALGORITHMIC DESIGN

- **IDEA Matrix.** Increase: investigator productivity. Drive: evidence-based conclusions. Educate: students on artifact structure. Accelerate: timeline reconstruction. Improve: integrity assurance via hashing. Deliver: CSV/HTML reports. Evaluate: browsing patterns (top domains, daily activity). Associate: downloads with their source pages. Ignore: cosmetics (GUI). Decrease: manual SQL work. Eliminate: risk of altering original evidence. Avoid: third-party dependencies.

- **Feasibility.** All operations are polynomial: SQL join with sort $O(m \log m)$, analysis passes $O(m)$, hashing $O(\text{file size})$. The problem is in P; there is no NP-hard component.

- **Function.** $x =$ History file, $y = f(x) = (h, T, D, K, L)$ where h is the hash, T the timeline, D the domain table, K the keyword hits and L the download list.

ANNEXURE B

LABORATORY ASSIGNMENTS ON PROJECT QUALITY AND RELIABILITY TESTING OF PROJECT DESIGN

- **Objects and morphisms.** Objects: History file, visit record, download record, report. Morphisms: acquire (file → copy + hash), parse (copy → records), analyse (records → statistics), report (statistics → files). The functions are composed sequentially: report ∘ analyse ∘ parse ∘ acquire.
- **Functional dependency graph.** report depends on analyse; analyse depends on parse; parse depends on acquire. No cycles; see the DFD in Fig.6.4.
- **Black-box test data.** Generated by `make_sample_db.py` with a fixed random seed (56 visits over 5 days, 3 downloads, one domain containing the keyword "bank").
- **Black-box test cases.** Listed in Table 9.1.

ANNEXURE C
PROJECT PLANNER

Week	Activity	Deliverable	Status
1	Study Chromium History schema, WebKit timestamps, NIST 800-86	Notes, design sketch	Done
2	Implement acquisition, parser, CLI; sample DB generator	`bhft.py` v0.1	Done
3	Analysis and reporting; pytest suite; live Edge test	`bhft.py` v1.0, tests	Done
4	README, GitHub repository, project report	Repository, report	Done

Table C.1: Project planner

ANNEXURE D

REVIEWERS COMMENTS OF PAPER SUBMITTED

Not applicable for this mini project.

ANNEXURE E
PLAGIARISM REPORT

To be attached after checking through the institute's plagiarism tool.

ANNEXURE F

TERM-II PROJECT LABORATORY ASSIGNMENTS

1. Review of design: the design was reviewed with the guide; the suggestion to add hash verification of the copy was incorporated.
2. Project workstation: Windows 11 laptop with Python 3.14, VS Code, Git and Microsoft Edge; setup took under 30 minutes.
3. Programming: completed as described in Chapter 8.
4. Testing: pytest suite of seven tests and a live-profile system test; results in Chapter 9.

ANNEXURE G

INFORMATION OF PROJECT GROUP MEMBERS

1. Name : Vishwajeet Tulse
2. Date of Birth :
3. Gender :
4. Permanent Address :
5. E-Mail :
6. Mobile/Contact No. :
7. Placement Details :
8. Paper Published : –

1. Name : Student Name 2
2. Date of Birth :
3. Gender :
4. Permanent Address :
5. E-Mail :
6. Mobile/Contact No. :
7. Placement Details :
8. Paper Published : –

1. Name : Student Name 3
2. Date of Birth :
3. Gender :
4. Permanent Address :
5. E-Mail :
6. Mobile/Contact No. :
7. Placement Details :
8. Paper Published : –

1. Name : Student Name 4
2. Date of Birth :
3. Gender :
4. Permanent Address :
5. E-Mail :
6. Mobile/Contact No. :
7. Placement Details :
8. Paper Published : –